Day 47: Deploying a Machine Learning Model with Flask

Beginner-friendly guide, complete Python project, code explanation, practical exercises, solutions, and quiz

1. Learning objectives

• Understand how Flask connects an application to a trained model.

• Understand routes, endpoints, HTTP methods, JSON requests, and responses.

• Build a prediction endpoint with input validation.

• Load a saved model once at application startup and reuse it.

• Test valid and invalid requests.

2. From Day 46 to Day 47

On Day 46, you trained, evaluated, saved, and reloaded a model. Today, you put that model behind a web API. A client sends input to a URL, and the API returns the model prediction as JSON.

Roman Urdu: Day 46 mein hum ne model train aur save kiya. Aaj Flask API banati hai jahan doosri application input send karti hai. API input validate karti hai, model se prediction leti hai, aur JSON response return karti hai.

3. Request-to-prediction workflow

Stage	What happens
Client	Sends an HTTP request containing JSON
Flask route	Receives the request at a URL endpoint
Validation	Checks the input structure and values
Model	Calculates a prediction using the loaded estimator
Response	Returns prediction or an error as JSON

4. Important concepts

• Flask: a lightweight Python framework for web applications and APIs.

• API: a defined interface that lets software systems communicate.

• Route or endpoint: a URL path handled by a function.

• GET: commonly used to retrieve information.

• POST: commonly used to send data to a server.

• JSON: a text format for structured data exchange.

• Status code: a number describing the result of an HTTP request.

5. Install and run the project

Install dependencies in the Python environment you use to run the script:

```
python -m pip install flask scikit-learn joblib numpy
```

Save the supplied project as Day-47-Flask-ML-API-Complete-Project.py, then run:

```
python Day-47-Flask-ML-API-Complete-Project.py
```

The first run trains and saves diabetes_model.joblib beside the script. The Flask development server then listens at http://127.0.0.1:5000. Open the root URL in a browser to check the status response.

6. Complete Python project

```
"""
Day 47: Flask Machine Learning Prediction API
Install: python -m pip install flask scikit-learn joblib numpy
Run: python Day-47-Flask-ML-API-Complete-Project.py
POST endpoint: http://127.0.0.1:5000/predict
```

```
Educational demo only. The diabetes dataset target represents disease
progression. This is not a medical diagnostic tool.
"""
from pathlib import Path
import joblib
import numpy as np
from flask import Flask, jsonify, request
from sklearn.datasets import load_diabetes
from sklearn.ensemble import RandomForestRegressor

MODEL_PATH = Path(__file__).with_name("diabetes_model.joblib")
app = Flask(__name__)

def train_and_save_model():
    """Train the sample model and save its fitted parameters."""
    dataset = load_diabetes()
    model = RandomForestRegressor(n_estimators=100, random_state=42)
    model.fit(dataset.data, dataset.target)
    joblib.dump(model, MODEL_PATH)
    print(f"Model saved: {MODEL_PATH}")

def load_model():
    """Load the saved model, training it first if needed."""
    if not MODEL_PATH.exists():
        print("Model file not found. Training model...")
        train_and_save_model()
    # Only load model files from trusted sources.
    return joblib.load(MODEL_PATH)

# Load the model once when the Flask application starts.
model = load_model()

@app.route("/", methods=["GET"])
def home():
    """Return API status and endpoint information."""
    return jsonify({
        "message": "Day 47 Flask ML API is running.",
        "prediction_endpoint": "POST /predict",
        "required_feature_count": 10
    })

@app.route("/predict", methods=["POST"])
def predict():
    """Validate a JSON payload and return a prediction."""
    data = request.get_json(silent=True)

    if not isinstance(data, dict) or "features" not in data:
        return jsonify({"error": "Send JSON containing a 'features' list."}), 400

    features = data["features"]
    if not isinstance(features, list) or len(features) != 10:
        return jsonify({"error": "Exactly 10 numeric features are required."}), 400

    valid = all(
        isinstance(value, (int, float))
        and not isinstance(value, bool)
        and np.isfinite(value)
        for value in features
    )
    if not valid:
        return jsonify({"error": "Every feature must be a finite number."}), 400

    input_array = np.asarray(features, dtype=float).reshape(1, -1)
    try:
        result = float(model.predict(input_array)[0])
    except (ValueError, TypeError):
        return jsonify({"error": "The model could not process this input."}), 400

    return jsonify({
        "prediction": round(result, 2),
        "model": "RandomForestRegressor",
        "feature_count": 10
    })

if __name__ == "__main__":
    # Local development server only. Use a production WSGI server in production.
    app.run(host="127.0.0.1", port=5000, debug=False)
```

7. Test the prediction endpoint

Send a POST request to http://127.0.0.1:5000/predict with a JSON body containing exactly 10 finite numeric features:

```
{
  "features": [
    0.038, 0.051, 0.062, 0.021, -0.044,
    -0.034, -0.043, -0.003, 0.019, -0.017
  ]
}
```

A successful response contains a prediction, model name, and feature count. The exact number depends on the model and input values.

```
{
  "prediction": 145.67,
  "model": "RandomForestRegressor",
  "feature_count": 10
}
```

8. Code walkthrough

MODEL_PATH: Builds a path to the model file beside the Python script.

train_and_save_model(): Loads the sample dataset, trains RandomForestRegressor, and saves it with joblib.dump().

load_model(): Trains the model if the file is missing, then loads the saved estimator.

Load once at startup: The model is loaded once when the app starts, instead of reading the file for every request.

@app.route('/'): Registers a GET endpoint for API status.

@app.route('/predict', methods=['POST']): Registers the endpoint that accepts prediction requests.

request.get_json(): Reads the JSON request body.

Validation: Checks that features is a list of exactly 10 finite numbers. Invalid input gets HTTP 400.

model.predict(): Runs inference. The array is reshaped into one row with ten feature columns.

jsonify(): Converts a Python dictionary into a JSON HTTP response.

9. Common errors

ModuleNotFoundError: Install the missing package in the active Python environment.

Connection refused: Check that the server is running and the URL/port are correct.

400 Bad Request: Send valid JSON with exactly 10 finite numeric values.

Missing model file: The script trains and saves it on the first run. Check write permissions if this fails.

Wrong feature order: Use the same feature order expected during training.

10. Practice tasks with solutions

Task 1: Send only five values. **Solution:** The API should return HTTP 400 and explain that exactly ten values are required.

Task 2: Send a string as one feature. **Solution:** Validation rejects it before the model receives the input.

Task 3: Why load the model once at startup? **Solution:** It avoids repeatedly reading the model file for every request and reuses the same loaded estimator.

Task 4: Rename the endpoint to /model/predict. **Solution:** Change the route decorator and update the client URL to match.

11. Revision quiz

1. What is Flask used for in this project?

2. Which HTTP method sends feature values?

3. Which function reads JSON request data?

4. What does jsonify() return?

5. Why validate input before calling model.predict()?

Answer key: 1. Build the web API. 2. POST. 3. request.get_json(). 4. A JSON HTTP response. 5. To reject malformed input and protect the prediction workflow.

12. Responsible deployment and references

This local Flask development server is for learning only. Production systems need a production WSGI server, HTTPS, secure configuration, access controls, request limits, logging, monitoring, and tested model versions. Only load serialized model files from trusted sources. The built-in diabetes dataset target represents disease progression. This project is not a diagnostic tool and must not be used for medical decisions.

- Flask documentation
- Flask Quickstart
- scikit-learn model persistence
- joblib documentation

Completion checklist: Run the script, confirm the model file is created, open the root endpoint, send a valid POST request, test an invalid feature count, and explain the request-to-prediction flow.